CMC VELLORE · DEPARTMENT OF ANAESTHESIA

Duty Rota Software
Developer Handbook

Architecture, workflows, algorithms and operations reference for the rota board web application

VERSION
1.0

DATE
17 October 2026

CODE BASELINE
main @ a231ddf

AUDIENCE
Development & handover team

STACK
FastAPI · PostgreSQL · TypeScript/Vite

COMPANION
Inspection Report 2026-10



Reference Handover Source: docs/handbook/DEVELOPER_HANDBOOK.html · Markdown version: docs/DEVELOPER_HANDBOOK.md

Contents

18 chapters · issue IDs (S1, B2, P1 …) refer to the Inspection Report and to `KNOWN ISSUE` comments in code

1 What the system does
2 Quick start
3 Repository layout
4 Architecture
5 Domain glossary
6 Data model
7 Monthly rota workflow
8 Core algorithms
9 Leave, units and imports
10 Historical data & analysis
11 Authentication and roles
12 API reference
13 Frontend guide
14 Testing
15 Deployment and operations
16 How-to recipes
17 Conventions and gotchas
18 Known issues & further docs

~41k	85	30	157
LINES OF CODE	API ROUTES	DATABASE TABLES	BACKEND TESTS (PASSING)

READ BEFORE CHANGING ANYTHING

The October 2026 inspection found critical security problems: a **public** GitHub repository containing credentials and a database dump (S1), a password-reset flow that lets anyone take over any account (S2), and 32 endpoints without authentication, including destructive ones (S3). Work through the “Immediate actions” in `docs/INSPECTION_REPORT_2026-10.md` first.

1 What the system does

Purpose, users and confirmed product decisions

The rota board of the Anaesthesia department (~220 doctors across several units and two campuses) uses this application to plan, check and publish the monthly on-call duty rota, and to analyse historical duty load.

① Members
Canonical names, aliases (spreadsheet spellings), designations and each doctor's default call level.

② Leave
Recorded manually or imported from the monthly leave spreadsheet; approved leave blocks duties.

③ Unit postings
Monthly Unit Management board: who is in which unit at which call level, plus DRP / SICU / Pain postings.

④ Rota generation
Stage 1 allocates every duty slot to a **unit**; stage 2 assigns a **person** (manual with suggestions, or safe auto-fill).

⑤ Review & publish
Warnings, overrides, fairness and swap requests are reviewed; the month is published with an approval note and exported to Excel.

⑥ Analysis
Historical data (Jan 2025 – May 2026) and published months feed a duty-load dashboard.

Confirmed product decisions

Source: `Plan/05_confirmed_decisions.md`

Decision	Implication for developers
Admin-only users	No staff self-service logins. Roles: rota board member, computer admin, superadmin.
Both import *and* creation	The MVP covers historical import/analysis and new monthly rota creation.
Excel output	Final rota and template exports are `.xlsx` (xlsxwriter).
≥ 24 h between two 24-hour duties	Hard rule: violations must not be publishable without an explicit override reason.
Configurable duty limits	Monthly limits live in the rule set (admin UI), not in code.
Drafts don't affect analysis	Only historical / published periods count in the dashboard.
Admin-editable mappings	Spreadsheet label → system key mappings are reviewed in the UI.
Trusted roster	The department name-list workbook is the reference for canonical names.

2 Quick start

Run the app locally in minutes

Prerequisites

Windows (helper scripts are PowerShell), Python 3.11+ (3.12 in Docker), Node 22, PostgreSQL 16 (local install or Docker Desktop).

One command

```
.\launch.bat        # or: powershell -ExecutionPolicy Bypass -File .\start.ps1
```

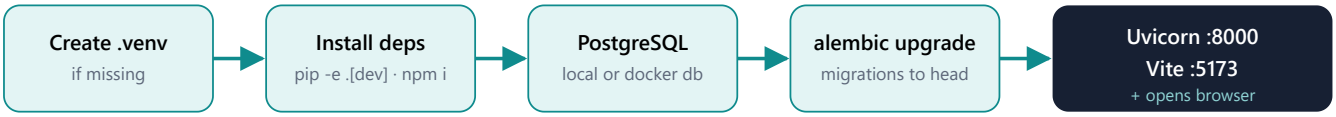


Figure 2.1 — What `start.ps1` does

Manual setup

```
# database
docker compose up -d db

# backend
cd backend
python -m venv .venv ; .\.venv\Scripts\Activate.ps1
pip install -e ".[dev]"
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# frontend (second terminal) — /api is proxied to 127.0.0.1:8000
cd frontend ; npm install ; npm run dev
```

Configuration (`.env` at repo root or `backend/`)

Variable	Default	Purpose
`DATABASE_URL`	`postgresql+psycopg://duty_rota:duty_rota@localhost:5432/duty_rota`	`postgres://` URLs are rewritten to the psycopg driver
`FRONTEND_ORIGIN`	`http://localhost:5173`	Added to the CORS allow-list
`APP_ENV`, `APP_NAME`	development	Informational only
`VITE_API_BASE_URL`	empty	Leave empty for same-origin (proxy / Railway)

FIRST LOGIN

On an empty database, `POST /api/v1/auth/seed-superadmin` creates `rotachief` with the default password from the planning docs. Change it immediately and read issues S2/S4 before deploying.

Source data the import features expect (not in git)

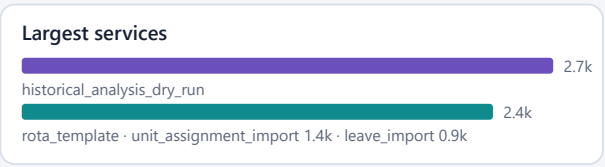
Path	Used by
`data/source/historical/*.xlsx`	Monthly rota workbooks — historical import, mapping scan
`data/source/historical/unitwise/*.xlsx`	Unitwise posting workbooks; `May 2026.xlsx` drives call-level prefill
`Plan/Data/ANAESTHESIA department doctors(namelist).xlsx`	Trusted roster — reconcile / reset
`Plan/Data/CMC_Duty_Analysis_v5.html`	Reference report for comparing historical totals

3 Repository layout

Where everything lives

backend/ · Python / FastAPI

`app/main.py`	app factory, CORS, static mount
`app/core/config.py`	settings from env
`app/db/session.py`	engine, Base, get_db
`app/api/v1/*.py`	20 routers (HTTP layer)
`app/services/*.py`	24 modules — all business logic
`app/models/*.py`	SQLAlchemy ORM (30 tables)
`app/domain/*.py`	call levels, duty types
`alembic/versions/`	17 migrations
`tests/`	157 pytest tests (SQLite)



frontend/ · TypeScript / Vite

`src/main.ts`	entire SPA · 9.4k lines
`src/services/api.ts`	typed client · 2k lines
`src/styles.css`	all styling · 4.8k lines
`vite.config.ts`	dev proxy /api → :8000

Root & documentation

`Plan/`	decisions, roadmap, dev log, user guide
`docs/`	algorithms, deployment, handbook
`Dockerfile, railway.json`	production image
`docker-compose.yml`	local containers
`start.ps1, launch.bat`	Windows launcher
`backups/ cred/ Debug/`	must leave git (S1)

Figure 3.1 — Repository map

LAYERING RULE

API modules stay thin (validation, auth dependency, error mapping); all logic goes into `services/`; persistence through `models/`; schema changes through Alembic.

4 Architecture

Runtime components, layers and deployment shapes

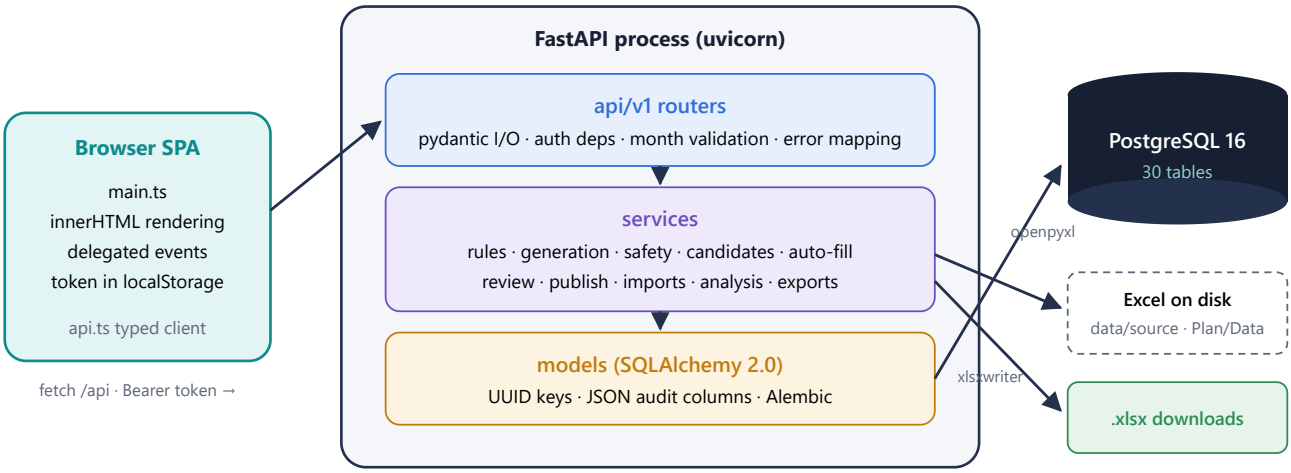


Figure 4.1 — Runtime architecture

API layer · `app/api/v1`

Declares request/response models, applies `Depends(current_user)` / `Depends(require_admin)` where present, validates `month=YYYY-MM` via `month_bounds()`, calls one service and maps `ValueError`, `RotaAssignmentError`, `RotaReviewError`, `RotaPublishError` to HTTP errors.

Service layer · `app/services`

All business rules. Services receive a `Session` and **commit themselves**. Many responses are plain dicts built by `*_to_dict` helpers.

Persistence

SQLAlchemy 2.0 typed ORM, UUID primary keys, naive UTC timestamps, JSON snapshots for audit. Alembic owns schema changes.

No background work

No job queue, cache or scheduler. Generation, auto-fill and historical import all run synchronously inside the HTTP request.

Deployment shapes

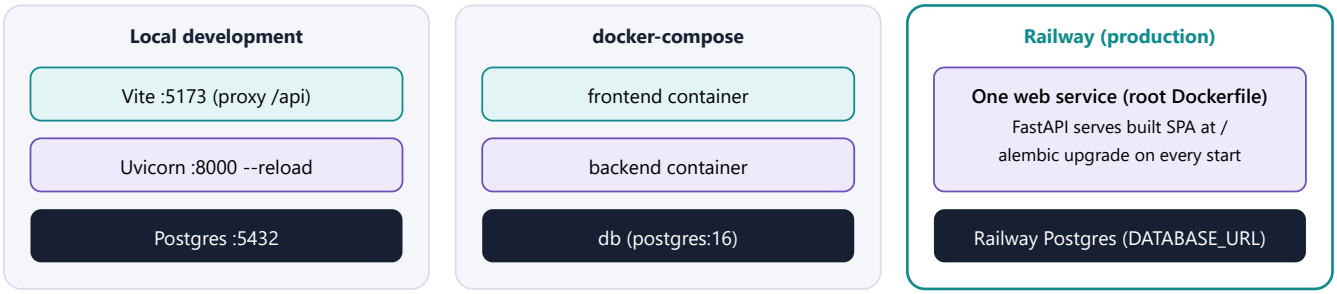


Figure 4.2 — The three ways the system runs

5 Domain glossary

The vocabulary used in code, UI and this handbook

Call-level ladder (junior → senior)

1ST_CALL	2ND_CALL	3RD_CALL	CO_4TH_CALL	4TH_CALL	5TH_CALL
junior PGs (+Co-1st)	senior PGs / SRs	SR / AP · groups A/B/C	co-4th	consultants	senior consultants

Normalised by services/rota_call_levels.normalize_call_level · special postings DRP / SICU / PAIN have no unit

Term	Meaning in this codebase
Member / Person	A doctor (`persons`). `active_status`: `active`, `archived`, `historical` (created by the historical importer).
Alias	Another spelling of a member's name seen in spreadsheets; used by every importer to resolve names.
Designation	Dated job title (e.g. ASSISTANT PROFESSOR) from the trusted roster.
Unit	Clinical unit (Unit I–VI, cardiac, neuro…) with `minimum_free_people` and optional per-call minimums.
Posting	`person_postings`: member X in unit Y at call level Z for a date range. Source `unit_board` is authoritative for new rotas.
Duty type	Key from `domain/duty_types.py`, e.g. `MAIN_1ST_24HR`, `RC_3RD_24HR`, `CB_PAEDS`, `MAIN_PAC_SR_A`, `CHAD`.
Duty family (group)	`main` (main campus), `cb` (Centenary), `rc` (Ranipet campus), `caesar`, `schell`, `floating`, `fifth_call`, `cart`, `pac` (pre-anaesthetic clinic), `shift`, `chad`, `ruhsa`, `paeds`, `neuro`.
Main 24-hour duties	`MAIN_24HR_KEYS` — counted in the headline "24hr" analysis totals.
Eligibility group	"Call cluster": a sub-group within a call level (e.g. 3rd Call A/B/C) used to restrict PAC duties.
Rule set	`RotaPhaseOneRules` JSON: per-duty rules, monthly limits, rest rules, staffing thresholds.
Rota period	One month (`rota_periods`), status `draft`, `published` or `historical`.
Generation scope	Units taking part in this month's generation; must be **locked** before generating.
Template / slot	`duty_slots` row (`source="phase4_template"`): a duty on a date, allocated to a unit, waiting for a person.
Forced allocation	Slot given to the least-damaging unit when every unit was hard-blocked; always needs review.
Vertical shuffle	Post-generation swap of units between same-duty slots on different days to reduce clustering.
Safety	Per-slot staffing status: `safe`, `needs_review`, `hard_blocked`.
Override	Reason text required to save an assignment that has warnings/errors.
Exchange	Swap request replacing the person on an assignment; approved or rejected later.
Eagle Eye export	Excel matrix of duty × date showing the allocated unit.

6 Data model

30 tables created by migrations 0001 – 0017

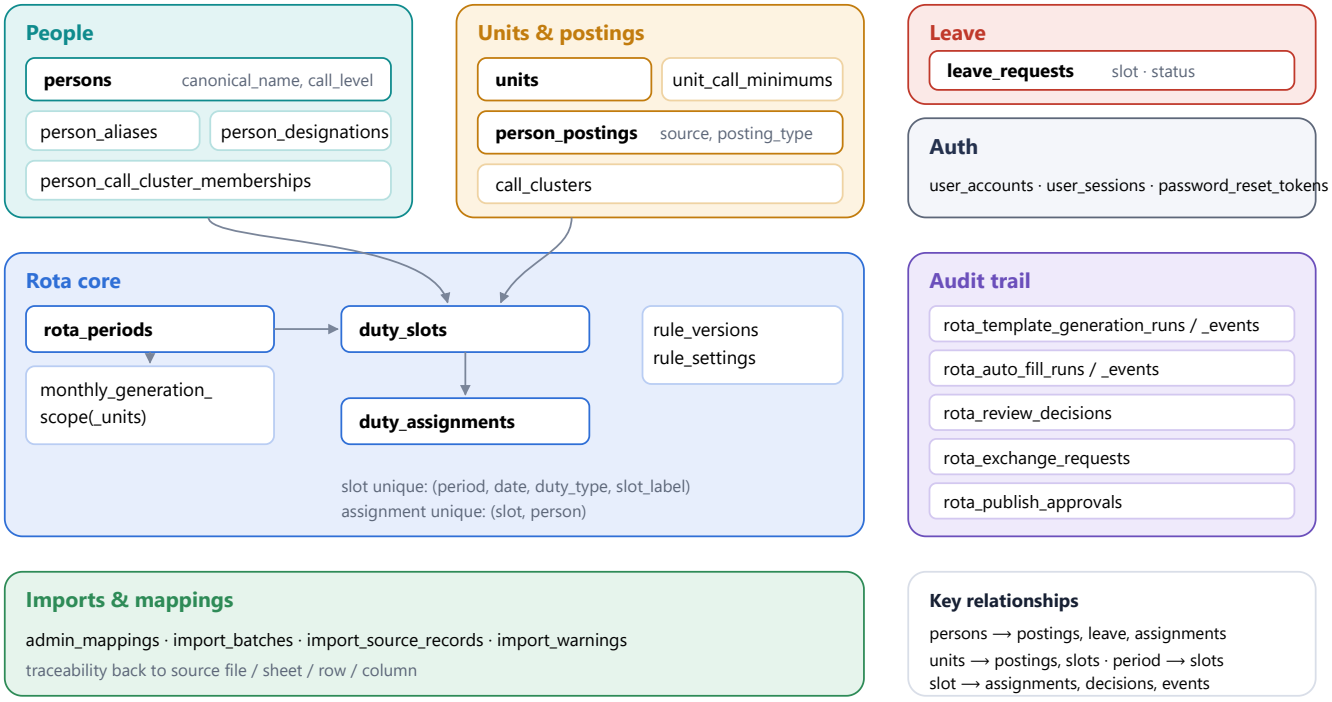


Figure 6.1 — Table groups and main relationships

Area	Tables	Notes
People	`persons`, `person_aliases`, `person_designations`	`canonical_name` unique; `alias` globally unique
Units & postings	`units`, `unit_call_minimums`, `person_postings`	posting source: `unit_board`, `historical_import`, `historical_analysis_import`
Leave	`leave_requests`	slot FULL_DAY/AM/PM/NIGHT; status **approved** = blocking, requested / imported_pending_review = warning, cancelled = ignored
Rota	`rota_periods`, `monthly_generation_scope(_units)`, `duty_slots`, `duty_assignments`	generated slot label = `"<UNIT_CODE>:primary"`
Audit	generation, auto-fill, review, exchange, publish tables	JSON `details` / `summary` snapshots with reasons
Rules	`rule_versions`, `rule_settings`, `call_clusters`, memberships	one version + one JSON setting in practice (B11)
Imports	`admin_mappings`, `import_batches`, `import_source_records`, `import_warnings`	source traceability
Auth	`user_accounts`, `user_sessions`, `password_reset_tokens`	token hashes only

7 The monthly rota workflow

From members to a published Excel rota

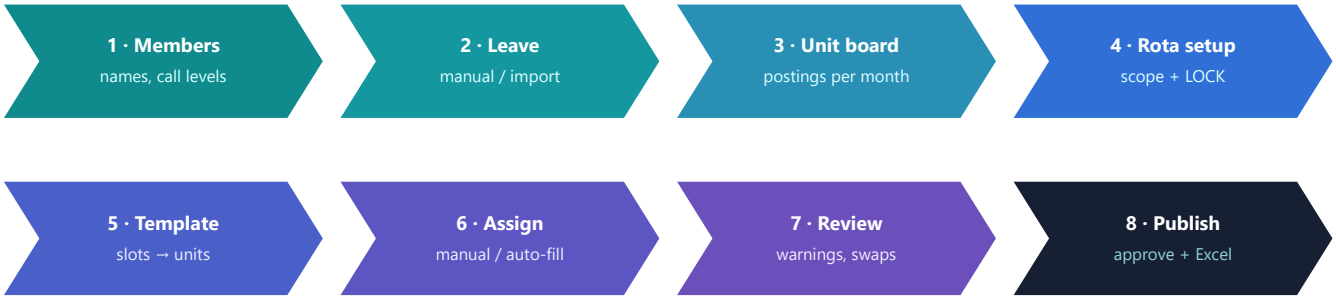


Figure 7.1 — The eight steps of a rota month

Step	Screen	Endpoints	Service
1 Members	`members`	`/admin/members/*`	`members.py`, `roster_reconciliation.py`
2 Leave	`leave`	`/leave/*`	`leave.py`, `leave_import.py`
3 Unit board	`units`	`/unit-management/*`, `/units`	`unit_management.py`, `unit_assignment_import.py`
4 Setup	`rota-setup`	`/rota-setup/month*`	`rota_setup.py`
5 Template	`rota-template`	`/rota-template/*`, `/rota-safety/month`	`rota_template.py`, `rota_safety.py`
6 Assign	day modal	`/rota-assignments/*`, `/rota-candidates/*`, `/rota-auto-fill/*`	`rota_assignment.py`, `rota_candidates.py`, `rota_auto_fill.py`
7 Review	`rota-review`	`/rota-review/*`	`rota_review.py`
8 Publish	`rota-publish`	`/rota-publish/*`	`rota_publish.py`

PUBLISHING DOES NOT FREEZE THE MONTH (B1)

After publishing, slots and assignments can still be changed and the export is rebuilt from live data. Treat this as a known gap until a publish lock is implemented.

What happens at each stage

1. **Setup** — `GET /rota-setup/month` gets or creates the `RotaPeriod` ("June 2026") and an empty scope, and reports per-unit readiness (members, call-level spread, leave). The board picks units and locks the scope.
2. **Generate** — `POST /rota-template/generate` requires a locked scope; by default removes previously generated slots (refuses if any is assigned) and creates one slot per day × mandatory active duty, each allocated to a unit (§8.1). Every decision becomes a generation event.
3. **Assign** — the day modal shows each slot's safety (§8.2) and ranked candidates (§8.4). Saving validates (§8.3); warnings need an override reason, *blocked* results cannot be saved. Safe auto-fill (§8.5) handles clean slots.
4. **Review** — open slots, template-review slots, staffing warnings/hard blocks and overrides; accept warnings with a note; request/approve/reject exchanges; workload and fairness tables.
5. **Publish** — checklist (§8.6); stores a `RotaPublishApproval`, sets the period to `published` and enables the final Excel export.

8 Core algorithms

Unit allocation, safety, validation, ranking, auto-fill and publishing



Figure 8.1 — Two-stage generation: units first, people second

8.1 Template generation — unit allocation

`services/rota_template.py` · `generate_empty_template` → `choose_balanced_unit` → `optimize_template_slot_layout`

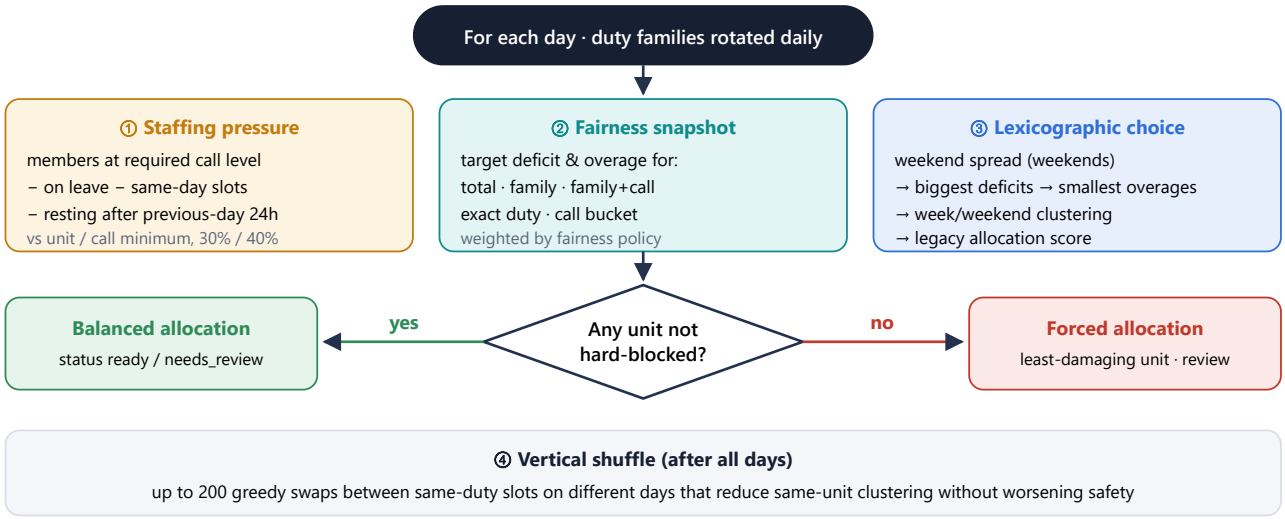


Figure 8.2 — Unit allocation decision flow for one (day, duty)

Fairness policies

equal_unit
EVERY INCLUDED UNIT GETS THE SAME TARGET SHARE

equal_people
SHARE PROPORTIONAL TO ACTIVE ELIGIBLE PEOPLE

hybrid (default)
≈ HALF EQUAL SHARE, HALF PROPORTIONAL

Weights: `allocation_target_weights`. Background: `docs/unit_slot_fairness_policy_modes.md`, `docs/unit_slot_imbalance_incident_report_2026_05.md`.

8.2 Slot safety

`services/rota_safety.py` · `slot_safety`, `safety_status`

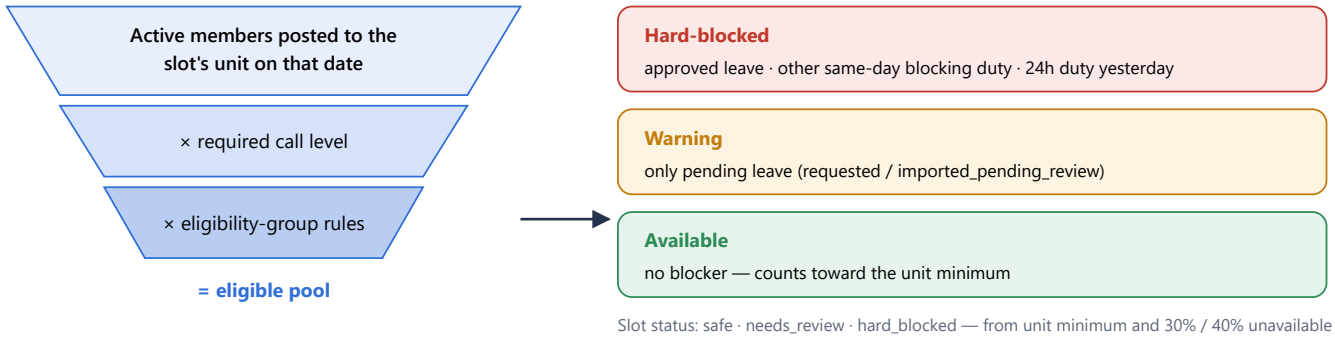


Figure 8.3 — How a slot's eligible people are classified

REST RULE GAP (B2 — REPRODUCED)

Rest is only checked **backwards one calendar day, inside the month**. A doctor with a 24h duty on the 13th can be given a 24h duty on the 12th with status `clear`; the 31st → 1st boundary is never checked; `minimum_gap_after_24hr_hours` only affects ranking and `post_24hr_blocks_next_day_elective` is not read anywhere.

8.3 Assignment validation

`services/rota_assignment.py` · `validate_assignment`, `assign_person_to_slot`

blocked — cannot be saved

slot full without "replace" · call-level mismatch · eligibility-group mismatch

needs_override — reason required

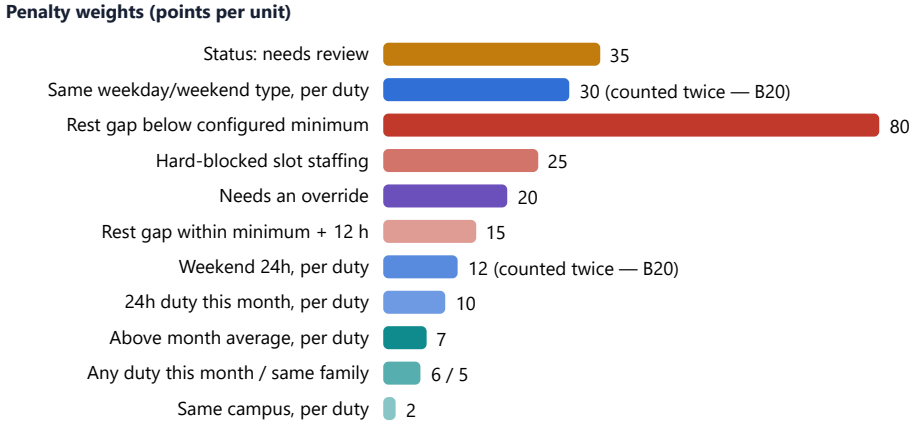
inactive member · not posted to the unit · leave / same-day / rest blockers · slot staffing warning or hard block · monthly limits (24h, weekend 24h, same family, same campus)

clear — saved directly

None of the above apply.

8.4 Candidate ranking

`services/rota_candidates.py` · `candidate_score` — lower score ranks higher



Sort order

1. candidate status
   eligible → review → blocked
2. total score (ascending)
3. name

Blocked candidates carry +1000.

Figure 8.4 — Relative weight of candidate score components

8.5 Safe auto-fill

Walks the month's slots in date order: skips filled slots; in strict mode skips slots whose required call level is ambiguous; assigns the first candidate that is **eligible**, validates **clear**, needs no override and (strict) matches the single required call level. Every decision is logged as an auto-fill event.

PERFORMANCE (P1) & ROBUSTNESS (B12)

The full month context is rebuilt per slot and each candidate is fully re-validated, so a full month can take minutes inside one HTTP request. The run row is saved as `completed` before work starts and each assignment commits separately.

8.6 Publish checklist

Blockers

- monthly scope not locked
- no generated slots
- open (unassigned) slots
- hard-blocked review items
- pending / needs-override exchanges

Warnings (need confirmation)

- review items not accepted with a note
- assignments saved with override reasons

Publish requires `confirm_warnings=true` when any are present.

9 Leave, units and imports

Every import follows preview → resolve → apply-safe-rows

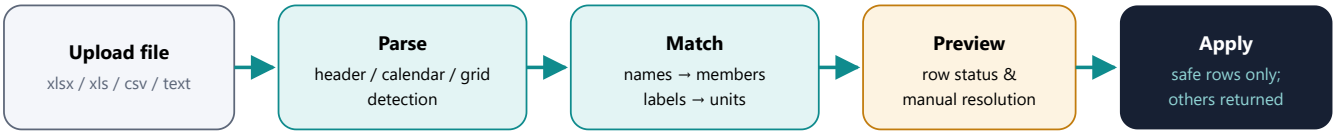


Figure 9.1 — Common import pattern (leave and unitwise imports)

Leave — `leave.py`, `leave_import.py`

- Manual leave only for `active` members; end date ≥ start date.
- Table sheets: header detected in the first 25 rows; columns recognised by name (name/person/doctor/member/staff, start/from/date, end/to, type, slot/session, status, notes). Otherwise a "wide calendar" layout with one column per day.
- Half-day markers in name cells become `leave_slot`; duplicates and already-recorded leave are flagged and skipped.

Leave status	Calendar	Effect on duties
`approved`	shown	HARD BLOCK
`requested`, `imported_pending_review`	shown	WARNING
`cancelled` or anything else (incl. typos — B13)	hidden	IGNORED

Unit Management — `unit_management.py`, `unit_assignment_import.py`

- Board postings (`source="unit_board"`) are what generation should use. Validation flags inactive members, members in more than one unit in a month, overlapping primary postings, missing unit or posting type.
- Unit settings: unit-level minimum free people plus per-call-level minimums (validated against that month's counts).
- Unitwise import: grid of units × posting sections, date ranges like `1-15`, `1st to 15th` or ISO ranges. Fuzzy matching thresholds: members 0.88, units 0.80, required lead over runner-up 0.04. Manual corrections are learned as aliases and unit mappings.

> **REPLACE EXISTING (B6)**
>
> "Replace existing" deletes every board posting that overlaps the month — including postings that also cover other months.

Admin mappings and the department roster

Mappings

`scan-historical` collects duty, unit and posting labels from the historical workbooks and seeds suggested targets (`classify_duty_label`); admins review them on the Mappings screen.

Roster

Reconcile: rename to roster spelling, merge duplicates, add aliases/designations, create missing members. **Reset**: destructive wipe — never on a live database (B4, S3). **Prefill call levels**: from the May 2026 unitwise workbook.

10 Historical data & analysis

Jan 2025 – May 2026 import pipeline and the dashboard

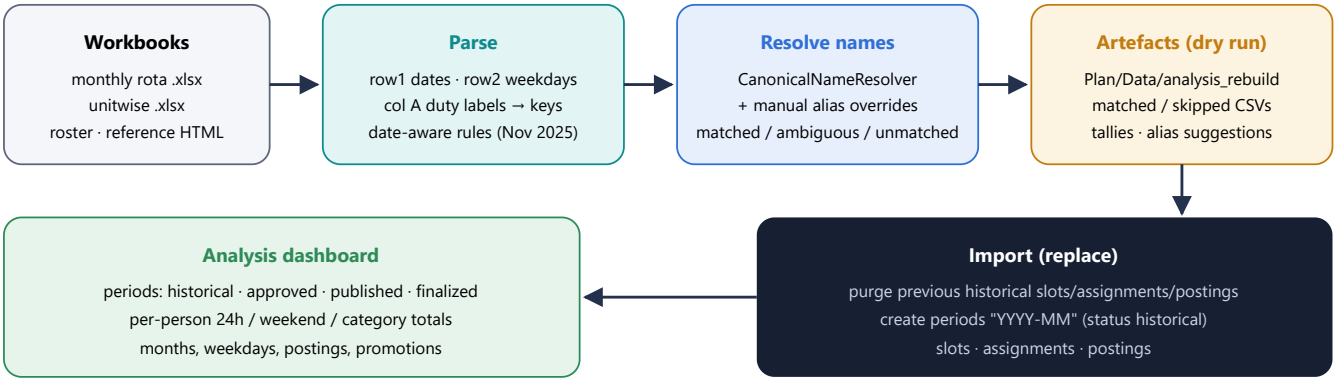


Figure 10.1 — Historical rebuild pipeline feeding the dashboard

```
cd backend
python -m app.services.historical_analysis_dry_run                    # dry run → CSVs
python -m app.services.historical_analysis_dry_run --apply-alias-suggestions
python -m app.services.historical_analysis_dry_run --import-matched-history
```

Name resolution order

#	Strategy	Confidence
1	Saved alias (unless ambiguous single token)	1.00
2	Compact exact match	1.00
3	Unique first name (≥ 4 letters)	0.93
4	Ordered token variant (prefix/suffix tokens; ties → ambiguous)	score
5	Unique first + last	0.97
6	Unique first + last initial	0.95

Alias decisions: `Plan/Alias/Alias_Resolution_Summary.md` (authoritative). Promotions: call levels are made monotonic over time (`enforce_monotonic_call_levels`).

PERIOD COLLISION (B3)

Historical periods are named `YYYY-MM`; Rota Setup looks periods up by dates and names them “Month YYYY”. For months covered by history, drafts land in the historical period and appear in analysis.

11 Authentication and roles

Bearer tokens, password hashing and the role model

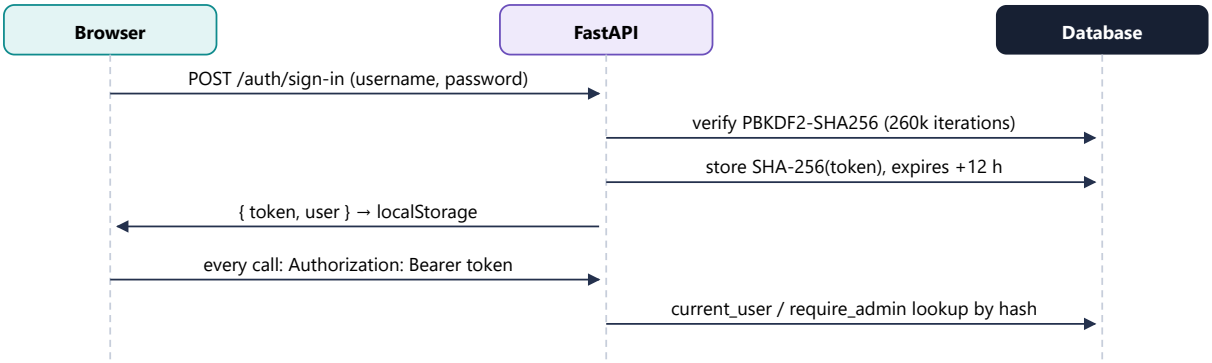


Figure 11.1 — Sign-in and authenticated requests

Role	UI	Backend dependency
rota_board_member	Workflow screens	`current_user` — can also publish, clear months and approve exchanges (S6)
computer_admin	+ Admin tools	`require_admin` — can create any account incl. superadmin (S6)
superadmin	+ Admin tools	`require_admin`

SECURITY STATUS

- **S2** — forgot-password returns the reset token; the login page auto-fills it → any account can be taken over.
- **S3** — members, mappings, historical import and analysis routes have no authentication at all.
- **S7** — no login throttling, no password policy on create/reset, sign-out is client-side only.

12 API reference

Generated from the running application — 85 route/method combinations

85	32	43	10
ROUTES	NO AUTHENTICATION	ANY SIGNED-IN USER	ADMIN ONLY



Figure 12.1 — Share of endpoints by required authentication

Month parameters are `month=YYYY-MM`. Interactive docs at `/docs` and `/redoc`. Errors: `{"detail": "…"}`; assignment validation failures return 409 with `{"detail": {"message", "validation"}}`.

Method	Path	Auth
System & auth		
GET	`/api/health`	NONE
GET	`/api/v1/metadata`	NONE
POST	`/api/v1/auth/sign-in`	NONE
POST	`/api/v1/auth/seed-superadmin`	NONE
POST	`/api/v1/auth/forgot-password`	NONE
POST	`/api/v1/auth/reset-password`	NONE
GET PUT	`/api/v1/auth/me`	USER
POST	`/api/v1/auth/change-password`	USER
GET POST	`/api/v1/auth/accounts`	ADMIN
GET	`/api/v1/diagnostics/summary`	ADMIN
Members		
GET POST	`/api/v1/admin/members`	NONE
PUT	`/api/v1/admin/members/{person_id}`	NONE
POST	`/api/v1/admin/members/{person_id}/aliases · /designations · /archive · /restore`	NONE
GET	`/api/v1/admin/members/audit · /dedupe-candidates · /invalid`	NONE
POST	`/api/v1/admin/members/merge · /auto-merge-duplicates · /cleanup-invalid`	NONE
POST	`/api/v1/admin/members/reconcile-trusted-roster · /prefill-call-levels`	NONE
POST	`/api/v1/admin/members/reset-from-trusted-roster` **destructive**	NONE
Mappings, imports, analysis		
GET POST	`/api/v1/admin/mappings`	NONE
GET	`/api/v1/admin/mappings/options`	NONE
PUT	`/api/v1/admin/mappings/{mapping_id}`	NONE
POST	`/api/v1/admin/mappings/scan-historical`	NONE
GET	`/api/v1/admin/imports/historical/status`	NONE
POST	`/api/v1/admin/imports/historical/run`	NONE
GET	`/api/v1/analysis/dashboard · /preflight · /manual-review`	NONE

Method	Path	Auth
Eligibility groups & rules		
GET POST	`/api/v1/admin/call-clusters`	ADMIN
PUT	`/api/v1/admin/call-clusters/{cluster_id}`	ADMIN
GET PUT	`/api/v1/admin/call-clusters/{cluster_id}/members`	ADMIN
GET PUT	`/api/v1/admin/rota-rules/phase-one`	ADMIN
Leave		
GET POST	`/api/v1/leave/requests`	USER
PUT	`/api/v1/leave/requests/{leave_id}`	USER
POST	`/api/v1/leave/requests/{leave_id}/cancel`	USER
GET	`/api/v1/leave/calendar · /pressure`	USER
POST	`/api/v1/leave/import-preview · /import-apply` (multipart)	USER
Unit management		
GET	`/api/v1/units`, `/api/v1/unit-management/month`	USER
PUT	`/api/v1/unit-management/units/{unit_id}/settings`	USER
POST	`/api/v1/unit-management/assignments`	USER
PUT DEL	`/api/v1/unit-management/assignments/{assignment_id}`	USER
POST	`/api/v1/unit-management/import-preview · /import-apply` (multipart)	USER
Rota		
GET	`/api/v1/rota-setup/month`	USER
PUT	`/api/v1/rota-setup/month/scope`	USER
POST	`/api/v1/rota-setup/month/clone-previous`	USER
GET	`/api/v1/rota-template/month · /allocation-statistics`	USER
POST	`/api/v1/rota-template/generate`	USER
DELETE	`/api/v1/rota-template/cache` (`clear_assignments`)	USER
GET	`/api/v1/rota-template/eagle-eye-export · /call-wise-export`	USER
GET	`/api/v1/rota-safety/month`	USER
GET	`/api/v1/rota-candidates/month · /slots/{slot_id}`	USER
POST	`/api/v1/rota-assignments/slots/{slot_id}/assign`	USER
DELETE	`/api/v1/rota-assignments/assignments/{assignment_id}`	USER
GET POST	`/api/v1/rota-auto-fill/month`, `/draft`	USER
GET	`/api/v1/rota-review/month`	USER
POST	`/api/v1/rota-review/slots/{slot_id}/decisions`	USER
POST	`/api/v1/rota-review/exchanges`, `/{id}/approve`, `/{id}/reject`	USER
GET	`/api/v1/rota-publish/month · /export`	USER
POST	`/api/v1/rota-publish/publish`	USER

13 Frontend guide

Framework-free TypeScript single-page app

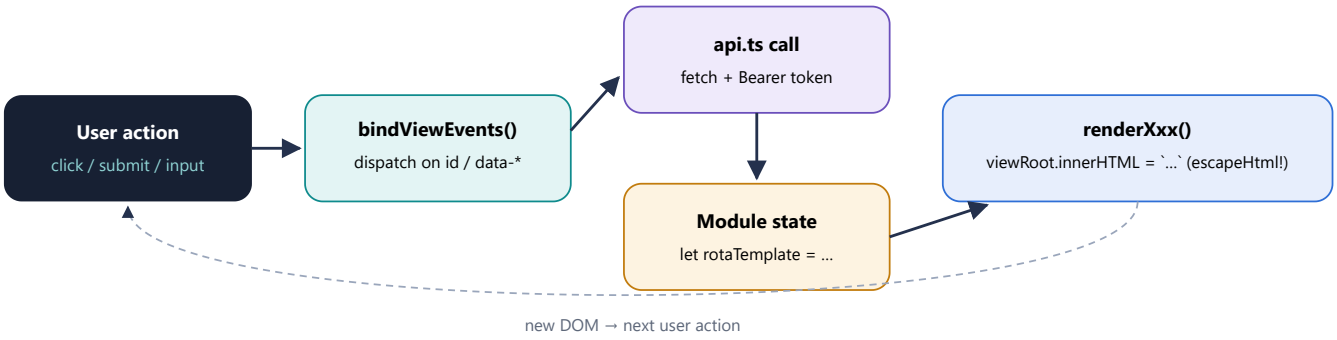


Figure 13.1 — Render/event loop in main.ts

Navigation map

Workflow screens	Admin tools (UI-hidden for board members)
overview · analysis · members · leave · units · rota-setup · rota-template · rota-review · rota-publish · user-guide	rota-rules · call-clusters · mappings · imports · accounts · diagnostics

Adding a screen

1. Add an entry to `NAV_ITEMS` and a branch in `navigateToView()`.
2. Write `async function renderX()`: `setHeader()`, load data via `services/api.ts`, set `viewRoot.innerHTML`.
3. Add handlers in `bindViewEvents()` keyed on ids / `data-*` attributes.
4. Escape every server string with `escapeHtml()`; use `preserveViewport()`, `setButtonLoading()`, `showToast()`, `confirmAction()`.

localStorage keys

auth token · template fast mode · overview mode · day-modal view · suggestions expanded

Build

`npm run build` (tsc + vite build) · type-check only: `npx tsc --noEmit`

FRONTEND KNOWN ISSUES

- **S5** unescaped names/labels in innerHTML
- **U1** expired 12 h session not handled
- **U2** document listeners duplicated after sign-out/in
- **U4** no URL routing — refresh returns to Overview

14 Testing

What is covered and what is not

```
cd backend
.\.venv\Scripts\python -m pytest -q -p no:cacheprovider     # 157 tests, ~45–75 s
cd ..\frontend
npx tsc --noEmit
```

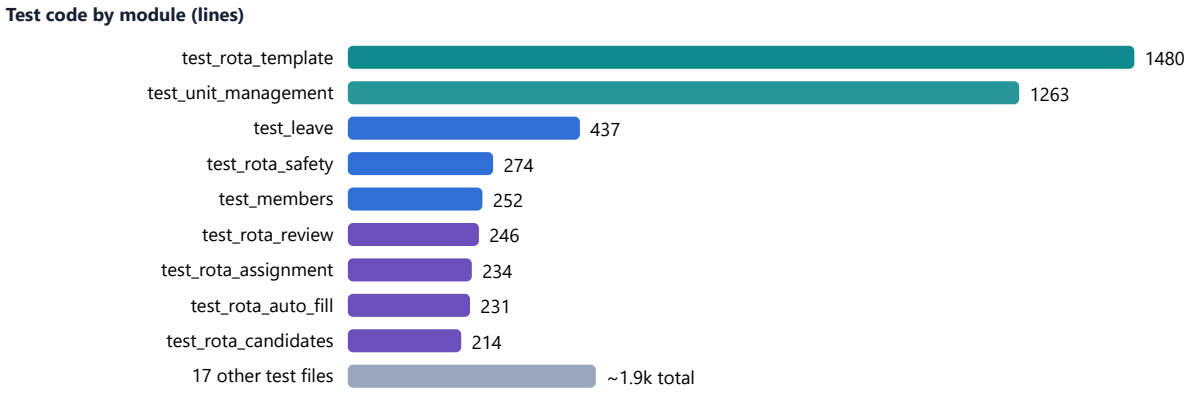


Figure 14.1 — Where the test effort is concentrated

Strengths

- Template generation regressions (fairness, rotation, statistics)
- Unit import parsing & matching
- Leave import formats; safety / review / publish flows

Gaps

- SQLite in-memory: foreign keys & PostgreSQL behaviour untested (B4 invisible)
- No auth coverage test; no frontend tests
- No CI pipeline

RECOMMENDED

Add a PostgreSQL test job (docker service) and a parametrised test asserting every non-public route returns 401 without a token.

15 Deployment and operations

Railway, migrations, backups, logs

Railway (current production)

Setting	Value
Services	One web service (root `Dockerfile`) + one PostgreSQL service
Variables	`APP_ENV=production` · `DATABASE_URL=${{Postgres.DATABASE_URL}}` · `FRONTEND_ORIGIN=https://<service>.up.railway.app`
Health check	`/api/health` (railway.json, timeout 300 s, restart on failure ×3)
Port	`${PORT:-8080}`
Migrations	`docker-entrypoint.sh` runs `alembic upgrade head` on every start

Migrations

```
cd backend
alembic revision -m "describe change"   # write upgrade/downgrade by hand
alembic upgrade head
```

Tests create tables from the models, not migrations — keep both in sync.

Backups

No automated backup exists. Schedule `pg_dump` against Railway (TCP proxy) and store dumps **outside** the repository (S1, D6).

Logs

stdout → Railway logs. Unit management and imports log timings at INFO; unhandled errors log the traceback.

PRODUCTION LIMITATIONS

- **D1** — historical import, mapping scan, roster reconcile/reset and call-level prefill need files that are not in the image (paths resolve under `/`).
- **P1/U5** — generation, auto-fill and imports run inside the request and can hit proxy timeouts.

16 How-to recipes

Common change patterns, step by step

➕ Add a new duty type

1. Add to `DUTY_TYPES` in `app/domain/duty_types.py` (and `MAIN_24HR_KEYS` if it counts as a main 24h duty). Rules merge it in automatically on next read.
2. Eligibility defaults: extend the `PAC_SUBDIVISION_RULES`-style dicts in `rota_rules.py`, or configure in Rota Rules.
3. Historical parser: teach `classify_duty_label` in `services/imports.py`.
4. Frontend groupings (`DUTY_GROUPS`, `DUTY_TEMPLATE_GROUP_ORDER/LABELS`) and export labels (`EAGLE_EYE_GROUP_LABELS`) for a new family.
5. Tests in `tests/test_rota_template.py`.

⚙ Add a rule setting

Extend the pydantic model in `services/rota_rules.py` **with a default** (old JSON must still validate) → use it in the service → add to `RotaPhaseOneRules` in `api.ts` → add to the form in `renderRotaRules()` and the `rule-*` submit handler.

🔌 Add an endpoint

Service function in `services/` → router function in `api/v1/` **with an auth dependency** → register router in `api/routes.py` if new → client function in `api.ts` → tests.

🗄 Add a column or table

Model change + Alembic migration + read models / `*_to_dict` + tests. If it references `persons`, `units`, `rota_periods` or `duty_slots`, also update merges and the roster reset (B4).

🩹 Fixing a known issue

Search the code for `KNOWN ISSUE (handbook <ID>)`, fix, add a regression test, then update the Inspection Report register and `Plan/development_log.md`.

17 Conventions and gotchas

Things that surprise new developers

#	Gotcha
1	Month strings are always `YYYY-MM`; `month_bounds()` validates them.
2	Many GET endpoints **create rows** on first access (rota period, scope, default rules, default clusters).
3	`normalize_call_level` folds Co-1st into 1st call and returns unknown strings unchanged.
4	`person_postings.posting_type` holds a call level for board postings but a free label for historical ones.
5	Template slot labels are `"<UNIT_CODE>:primary"`; uniqueness is per (period, date, duty, label).
6	Datetimes are naive UTC; the frontend's default month uses UTC (`toISOString`).
7	Services commit internally; composing services in one request does not give one atomic transaction.
8	Dataset specifics are hard-coded (roster sheet names, May 2026 workbook, manual alias overrides, 2026-03-01 effective date) — search for `2026` before reusing code for a new year.
9	Short non-24h duties default to 08:00→08:00 windows (B9) — check rule durations when touching timings.
10	The frontend hides admin tools, but backend protection is per-endpoint and incomplete (S3).

18 Known issues & further documentation

Summary of the October 2026 inspection

3	8	15	18
CRITICAL	HIGH	MEDIUM	LOW

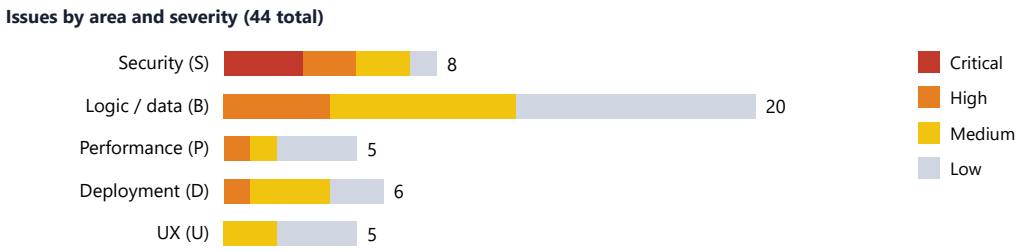


Figure 18.1 — Inspection findings (full register: docs/INSPECTION_REPORT_2026-10.md)

Critical and high issues

ID	Severity	Issue
S1	CRITICAL	Public GitHub repo contains credential screenshots and a DB dump with password hashes
S2	CRITICAL	Forgot-password returns the reset token → anyone can take over any account
S3	CRITICAL	32 endpoints without authentication, incl. roster reset, merges and historical import
S4	HIGH	Hard-coded default superadmin; public seed endpoint
S5	HIGH	Stored XSS via unescaped names/labels; token readable from localStorage
B1	HIGH	Published months not locked; final export rebuilt from live data
B2	HIGH	24-hour rest rule enforced only backwards one day (reproduced)
B3	HIGH	Historical and live rota periods collide for the same month
B4	HIGH	Roster reset / merges fail on PostgreSQL due to unhandled foreign keys
P1	HIGH	Auto-fill and candidate ranking re-query the whole month per candidate
D1	HIGH	Import / roster features cannot work in the Docker image

Further documentation

Document	Content
`docs/INSPECTION_REPORT_2026-10.md`	Full issue register, details, remediation roadmap
`Plan/README.md`, `Plan/08_new_session_checklist.md`	Planning index; what to read before changing behaviour
`Plan/01…07_*.md`	Product understanding, architecture, domain rules, decisions, principles, roadmap
`Plan/development_log.md`	Dated log of every change and verification — keep updating it
`Plan/Website complete user guide.md`	End-user guide for the rota board
`Plan/Rota generator/`, `leave management engine/`, `unit management engine/`	Feature plans
`docs/rota_*.md`, `docs/unit_*.md`	Algorithm research, validation, fairness policies, incident report